BMI / CS 771 Fall 2026:
Homework Assignment 2

October 2026

1 Overview

In this assignment, you will use PyTorch to design, implement, train, and evaluate deep neural networks for image classification and object detection. This is a team assignment that requires cloud computing. Each team should have 2–3 students unless otherwise approved by the instructor. The assignment is worth 12 points, with up to 4 additional bonus points. Detailed instructions and grading criteria are provided in Sections 3 and 4.

2 Setup

- We recommend using *pip* to manage your packages. Other popular options include *Conda* and *uv*.
- The following packages are required: PyTorch ($\geq$2.0 with GPU support), OpenCV ($\geq$3), NumPy, gdown, torchvision, pycocotools, PyYAML, Pillow, and TensorBoard. You are responsible for installing them.
- You may install additional common packages that are helpful for your implementation. **Describe any additional packages required for your implementation in your writeup.**
- You can debug your code and run small-scale experiments on CPUs. However, training a neural network on CPUs is computationally expensive, so GPU computing is required for this assignment. Please set up your team's cloud instance. **Remember to shut down the instance when it is not in use!**
- You will need to download datasets for both parts of this assignment. We have included scripts for downloading and unpacking them, assuming all dependencies are installed. See Sections 3 and 4.
- To complete the assignment, you will need to fill in the missing code in: *./code/cls_libs/modeling.py* and *./code/det_libs/modeling.py*

- Your submission must include your code, results, the required trained model checkpoints, and a writeup. See Sections 3 and 4 for details. You can generate the submission archive using:
python ./zip_submission.py

- An autograder will be used to grade most of the assignment. **Please follow the instructions closely.**

3 Part I: Image Classification

Image classification is a fundamental computer vision task that assigns category labels to input images. In this part, you will design, train, and evaluate convolutional networks and vision Transformers for scene recognition.

Dataset: For image classification, you will use MiniPlaces, a scene recognition dataset developed by MIT. It contains 120K images from 100 mutually exclusive scene categories, split into 100K images for training, 10K for validation, and 10K for testing. Download the dataset to ./data (see below). Performance will be measured using top-1 and top-5 accuracy on the validation set. For more details, see the dataset's GitHub page: `https://github.com/CSAILVision/miniplaces`.

Downloading the Dataset: We have included a script to download and unpack the dataset. Run
sh ./download_cls_dataset.sh

After a successful download, the data folder will contain two subfolders, "images" and "objects," and two text files, "train.txt" and "val.txt," in addition to data.tar.gz.

Requirements: Train your model using only the training set. Do not use validation labels for training. ImageNet pretrained weights are prohibited unless explicitly permitted below.

3.1 Implementation Overview

Code Organization: The helper code is organized into three parts.

- Source code for the input pipeline, model architectures, and training and inference utilities is located under ./code/cls_libs/.

- Configuration files (in YAML format) for the model architecture, training, and inference are stored under ./code/configs/.

- Training and evaluation scripts are provided in ./code/train_cls.py and ./code/eval_cls.py.

To view usage instructions for the training and evaluation scripts, run the following commands from ./code:
python ./train_cls.py --help
python ./eval_cls.py --help

Parameter Configuration: *cls_libs/config.py* specifies defaults for the model type, input processing, data loader, and training hyperparameters. You can override these defaults in a separate YAML configuration file. This allows you to change the model type or training settings without editing the source code. **See ./code/configs/cls_miniplaces_cnn.yaml for an example.** Separating implementation from configuration helps you manage experiments.

Hints: You can configure the code to run on a CPU during local development. Although not officially supported, you can also use the MPS backend to run on a compatible Mac GPU.

Please review the helper code before attempting this assignment.

3.2 Convolutional Neural Networks

3.2.1 A Simple Convolutional Neural Network

An Example CNN (not graded): We will start by training a deep network from scratch. No coding is needed for this step; we provide a data loader and a simple network. From ./code, run
python ./train_cls.py ./configs/cls_miniplaces_cnn.yaml

A GPU is required for training. Training may take some time and is expected to yield a model with over 40% top-1 accuracy on the validation set. **Run training inside a terminal multiplexer such as tmux so that the process continues if your SSH connection is interrupted.** You can use
watch -n 0.1 nvidia-smi
to monitor GPU utilization and memory consumption.

Monitor Training: Training loss, learning rate, training and validation accuracy, and model checkpoints are saved under ../logs. Each run creates a separate experiment folder whose name includes the configuration name and a timestamp. In the commands below, *exp_timestamp* denotes this folder. You can monitor the logged metrics using
tensorboard --logdir=../logs/exp_timestamp

Evaluate a Trained Model: Once training is complete, the final model is saved as ../logs/exp_timestamp/model_final.pth.tar. You can evaluate it by passing the experiment folder to the evaluation script:
python ./eval_cls.py ./configs/cls_miniplaces_cnn.yaml ../logs/exp_timestamp/

Resume Training from a Checkpoint: Training may be interrupted, for example, when using a spot instance. The training script saves epoch checkpoints so that you can resume using
python ./train_cls.py ./configs/cls_miniplaces_cnn.yaml
--resume=../logs/exp_timestamp/epoch_xxx.pth.tar

Recommended Practices: Copy the ../logs folder to your local machine and use TensorBoard locally to plot the curves needed for your writeup. This avoids the need to set up a TensorBoard server in the cloud. *You are responsible for managing experiment runs and their logs.* You can remove older experiments to save storage space once you have preserved the results you need. **Although this step is not graded, record the training curves and evaluation results. You will need them later.**

3.2.2 Design, Training, and Evaluation

Design Your Own Convolutional Network (2 Pts): Next, we will try to improve on the simple convolutional network. The goal is to design a network that balances accuracy and efficiency for scene recognition. There are several ideas you can explore. For example, adding more layers [10] may improve performance, but it may also make training less stable. You can also add batch normalization [6], parallel branches [11], or skip connections [4]. You can also adjust the training settings, such as the initial learning rate, optimizer, weight decay, number of epochs, and data augmentations, in the YAML configuration file. *Implement your network in cls_libs/modeling.py and use train_cls.py for training.* **In your writeup, describe (a) your model design, (b) the training scheme, and (c) your results. Include your configuration file, training logs (excluding intermediate checkpoints), and final checkpoint in the "results" folder of your submission.**

Fine-Tune a Pretrained Model: Next, we will fine-tune an 18-layer residual network (ResNet-18) pretrained on ImageNet [4], using a reduced learning rate and a small number of epochs. ImageNet pretrained weights are permitted for this step. The implementation is included in the helper code. Run
python ./train_cls.py ./configs/cls_miniplaces_resnet18.yaml

Compare and Discuss Your Results (1 Pt): How does your best model compare with (1) the provided simple network and (2) the fine-tuned ResNet-18? **Compare their training curves and evaluation results in your writeup.**

3.3 Vision Transformers

3.3.1 Understanding Self-Attention and Transformer Blocks

Self-Attention (1 Pt): Self-attention is a key component of Transformers [14]. We briefly review the idea introduced in lecture. Self-attention computes a

weighted average of input features, with weights determined by similarities between their query and key representations. Let $\mathbf{Z}^0 \in \mathbb{R}^{T \times D}$ denote a sequence of T tokens, each represented by a D-dimensional feature vector. The projection matrices $\mathbf{W}_Q \in \mathbb{R}^{D \times D_q}$, $\mathbf{W}_K \in \mathbb{R}^{D \times D_k}$, and $\mathbf{W}_V \in \mathbb{R}^{D \times D_v}$ produce the query, key, and value representations $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$, respectively, where $D_k = D_q$:

$$\mathbf{Q} = \mathbf{Z}^0\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{Z}^0\mathbf{W}_K, \quad \mathbf{V} = \mathbf{Z}^0\mathbf{W}_V. \tag{1}$$

The output of self-attention is given by

$$\mathbf{S} = \text{softmax}\left(\mathbf{Q}\mathbf{K}^T/\sqrt{D_q}\right)\mathbf{V}, \tag{2}$$

where $\mathbf{S} \in \mathbb{R}^{T \times D_v}$ and softmax is applied *row-wise*. Multihead self-attention (MSA) performs several self-attention operations in parallel and combines their outputs. **Implement MSA by filling in the missing code in the Attention class in cls_libs/modeling.py.** Helper code is provided in this class.

Transformer Block with Local Self-Attention (1 Pt): A main advantage of MSA is its ability to integrate context across the full sequence, but this comes at a computational cost. Standard MSA has a computational complexity of $O(T^2D + D^2T)$, making it expensive for long sequences derived from high-resolution images. In this assignment, you will implement a simpler alternative: local window self-attention [1, 7]. Restricting attention to windows of $W \times W$ tokens reduces the computational complexity to $O(W^2TD+D^2T)$, where $W^2 \ll T$. **Implement a Transformer block that supports local self-attention in the TransformerBlock class.** Follow the Pre-LN Transformer design [15] and include stochastic depth regularization [5]. To aid your implementation, we provide helper functions, including *window_partition* and *window_unpartition*, in cls_libs/modeling.py. See the code and comments for details.

3.3.2 Design, Training, and Evaluation

Design Your Own Vision Transformer (2 Pts): With self-attention and Transformer block implemented, you are ready to design a vision Transformer [2]. **Implement your model in the SimpleViT class.** To train it, run
python ./train_cls.py ./configs/cls_miniplaces_vit.yaml

This configuration uses the AdamW optimizer [9], with a lower learning rate and greater weight decay than the simple CNN configuration. With the default settings, the model is expected to perform similarly to the simple convolutional network. You are encouraged to adjust the default parameters of **SimpleViT** and the training hyperparameters to improve performance.

Vision Transformers may take substantially longer to train than a convolutional network, so extensive tuning is not required. **In your writeup, describe your vision Transformer design, training scheme, and results. Include your configuration file, training logs (excluding intermediate checkpoints), and final checkpoint in the "results" folder of your submission.**

4 Part II: Object Detection

Object detection is a central computer vision task that identifies and localizes object instances in an image. In this part, you will implement FCOS, a fully convolutional one-stage object detector. The conference version of FCOS [12] is listed in the course readings, but this assignment follows the **journal version** [13], which introduces improved design choices and provides more detailed explanations. Although FCOS is conceptually simple, its implementation requires care to ensure efficient training.

Dataset: For object detection, you will use PASCAL VOC 2007. It contains approximately 10K images with category labels and bounding box annotations for 20 object categories. The dataset is split into approximately 5K images in the training and validation (trainval) set and 5K in the test set. Download the dataset to ./data (see below). Performance will be measured using mean average precision (mAP) at an intersection-over-union (IoU) threshold of 0.5 on the test set. For more details, see [3].

Downloading the Dataset: We have included a script to download and unpack the dataset. Run
sh ./download_det_dataset.sh

After a successful download, the data folder will contain a subfolder named "VOCdevkit," along with VOCtest_06-Nov-2007.tar and VOCtrainval_06-Nov-2007.tar.

Requirements: Train your model using only the trainval set. Do not use test set for training. **Using an open source implementation of FCOS is not permitted in this assignment**, including the one in torchvision. Feel free to take some inspiration from an existing implementation.

4.1 Implementation Overview

Code Organization: The helper code is organized similarly to Part I.

- Source code for the input pipeline, model architectures, and training and inference utilities is located under ./code/det_libs/.
- Configuration files (in YAML format) for the model architecture, training, and inference are stored under ./code/configs/.
- Training and evaluation scripts are provided in ./code/train_det.py and ./code/eval_det.py.

The FCOS Model: FCOS predicts an object candidate, including its category label and bounding box, at each spatial location on the feature maps. The box is represented by offsets from that location to its four sides. This formulates object detection as a dense image labeling problem. As shown in Figure 1, the

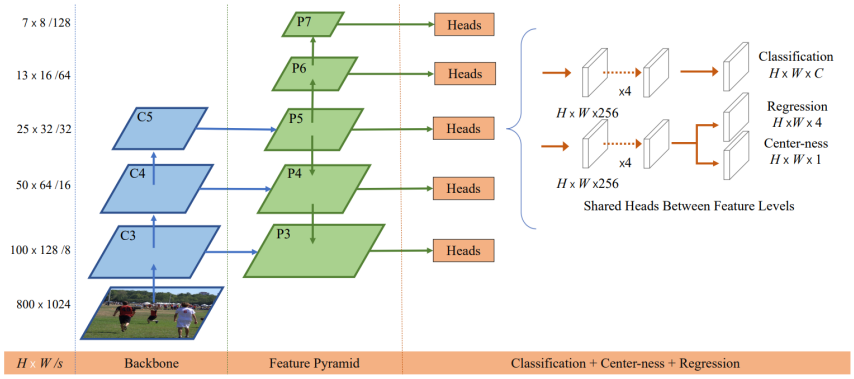


Figure 1: Overview of FCOS from [13]. The model consists of a backbone network, a feature pyramid, and shared classification / regression heads.

model consists of (a) a backbone, such as ResNet [4], that extracts image features at multiple scales; (b) a feature pyramid network (FPN) [8] that refines these features; and (c) shared classification and regression heads that predict object labels and boxes from the refined feature maps.

Input Pipeline (./code/det_libs/dataset.py and ./code/det_libs/transforms.py). Unlike image classification, object detection requires predicting structured outputs, such as bounding box coordinates, that depend on the image's spatial layout. Any data augmentation that changes pixel locations, such as flipping, cropping, or resizing, must therefore update the corresponding bounding boxes. Images within the same mini-batch may also have different sizes and aspect ratios, which makes efficient batching more challenging.

The helper code provides a standard input pipeline for training object detection models. The *VOCDetection* class loads and processes the VOC dataset. The *GeneralizedRCNNTransform* class implements common data augmentations, including horizontal flipping and scale jittering. After augmentation, images are padded to a common size so that they can be batched.

Model Architecture (./code/det_libs/modeling.py). This file implements FCOS, including the backbone, FPN, and prediction heads. The classification and regression heads are implemented in *FCOSClassificationHead* and *FCOSRegressionHead*, respectively; the remaining components are encapsulated in the *FCOS* class. Several critical pieces are missing. **You will need to complete them for the model to function properly.**

Training, Inference, and Other Utilities (other files under ./code/det_libs). These files provide training and inference pipelines, loss functions, visualization utilities, and other supporting functions. Two key components are (a) the training and evaluation functions in *./code/det_libs/engine.py* and (b) the centralized

parameter configuration in *./code/det_libs/config.py*. The configuration mechanism is similar to Part I, with more detailed control over the model architecture.

Training and Evaluation Interface: The helper code provides the following training and evaluation scripts. Run these commands from ./code. *Complete the missing training and inference code before running the scripts.*

- To train the model, run
  python ./train_det.py ./configs/det_voc_fcos.yaml

- By default, training logs and checkpoints are saved in a separate experiment folder under *../logs*. The folder name ends with a timestamp. You can monitor the training metrics using TensorBoard. As in Part I, we recommend copying the logs to your local machine and plotting the curves there.
  tensorboard --logdir=../logs/your_exp_folder

- To evaluate a trained model, run
  python ./eval_det.py ./configs/det_voc_fcos.yaml ../logs/your_exp_folder

- Evaluation automatically selects the final model checkpoint, if available, or the latest epoch checkpoint in the experiment folder. It saves *eval_results.json* in that folder. Use "-e" to select an epoch checkpoint or "-v" to save detection visualizations in its viz/ subfolder. See eval_det.py for details.

- The helper code also supports resuming training from a previous epoch checkpoint. This is usually unnecessary for VOC because an efficient implementation should finish training in under half an hour.
  python ./train_det.py ./configs/det_voc_fcos.yaml
  --resume=../logs/your_exp_folder/epoch_xxx.pth.tar

Hints: As in Part I, you can run the code on a CPU or a compatible Mac GPU by specifying the device in the configuration file.

Please review the helper code before attempting this assignment.

4.2 Model Inference

Decoding Objects (2 Pts): First, implement the FCOS inference procedure by completing the *inference* method of the *FCOS* class in det_libs/modeling.py. The comments provide a high-level outline, but the implementation details require care. To test your inference implementation, use the pretrained FCOS model with a ResNet-18 backbone and its configuration file, provided under ./pretrained in the assignment directory. From ./code, run
python ./eval_det.py ../pretrained/voc_res18.yaml ../pretrained/voc_res18.pth.tar

Inference should take a few minutes and achieve an mAP@0.5 of approximately 60.9%. If your inference time is much longer or your mAP differs substantially,

review your implementation. **Include the mAP score, inference time, and sample detection results in your writeup. Use the "-v" option during evaluation to generate visualizations.**

4.3 Model Training

Training with the Multitask Loss (3 Pts): Next, implement the FCOS training procedure. Its key component is a multitask loss that combines classification, regression, and centerness losses. Computing this loss requires assigning targets, such as positive and negative labels and box offsets, to feature map locations using the ground truth object annotations. The helper code handles most of the training procedure, but you must complete the *compute_loss* method of the *FCOS* class. This implementation can be challenging; the code comments provide some hints.

Your implementation must be **efficient**. An inefficient implementation can substantially increase training time. As a reference, training on VOC 2007 with the default configuration (ResNet-18 for 10 epochs) should take well under 30 minutes on a single T4 GPU. *Full credit will be awarded only if training completes within a reasonable time and the mAP scores are consistent with the reference implementation.*

Once your implementation is complete, train your own FCOS model using the instructions above. Refer to the pretrained model's configuration file and the journal paper for guidance on training settings. You are also encouraged to experiment with settings such as the learning rate and data augmentation.

To reduce turnaround time, we recommend starting with a lightweight model and/or a shortened training schedule when exploring hyperparameter settings. Once you identify a promising configuration, you can retrain using a more powerful backbone (e.g., ResNet-34) and the full training schedule.

In your writeup, include (a) your model's training curves, (b) test-set mAP scores, (c) sample detections on the test set, and (d) key implementation details and training settings. Include your configuration file, training logs (excluding intermediate checkpoints), and final checkpoint in the "results" folder of your submission.

4.4 Bonus: Extending the Detector

FCOS with a Swin Transformer (Bonus 2 Pts): The current implementation supports only residual networks as the backbone. To earn bonus credit, extend FCOS to support a Swin Transformer backbone and demonstrate the results. **You must (1) describe your implementation in the writeup and include the code in your submission, and (2) present training curves and evaluation results in the writeup.**

Training and Inference on the COCO Dataset (Bonus 2 Pts): You can also earn bonus credit by extending training and evaluation to the COCO

dataset. This requires downloading the dataset, modifying *dataset.py* and any other necessary helper code, and creating a new configuration file. To reduce experimentation time, we recommend using **a lightweight backbone** (e.g., ResNet-18) and **a small number of epochs** (e.g., 4). You can also use a subset of the training data to further cut down the training time. Extensive hyperparameter tuning is not required, as long as you can demonstrate the feasibility of your implementation. **You must (1) describe your implementation in the writeup and include the code in your submission, (2) describe your experimental procedure, including data preparation, in the writeup, and (3) report accuracy and efficiency in the writeup.**

5 Writeup

For this and all other assignments, submit a project report in PDF format. Every team member must submit the same report. For teams with more than one member, **clearly identify each member's contributions**. Describe your methods, explain your design and implementation choices, and present and discuss your results. Follow the detailed writeup requirements in each part of this assignment. You may also discuss additional experiments or other relevant information.

6 Submission

Follow the submission instructions below. Failure to do so will result in a 10% deduction. Your submission folder must contain the following directories:

- code/ — all code for this assignment.
- writeup/ — your project report in PDF format.
- results/ — your results, including configuration files, training logs, and the required final model checkpoints.

Use relative paths in your code, as the starter code does. Absolute paths (e.g., /user/classes/proj1) may cause your code to fail on the grading system and result in lost points. **Do not submit the original data/, logs/, or models/ directories.** Instead, copy the required logs and model checkpoints into results/. Include required training curves and evaluation results in your writeup. Submit your project as a single ZIP file through Canvas. You can create the archive using *python zip_submission.py*.

References

[1] K. M. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Q. Davis, A. Mohiuddin, L. Kaiser, et al. Rethinking attention with performers. In *ICLR*, 2020.

[2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

[3] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman. The pascal visual object classes (voc) challenge. *IJCV*, 88(2):303–338, 2010.

[4] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.

[5] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger. Deep networks with stochastic depth. In *ECCV*, 2016.

[6] S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.

[7] Y. Li, H. Mao, R. Girshick, and K. He. Exploring plain vision transformer backbones for object detection. In *ECCV*, 2022.

[8] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017.

[9] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2018.

[10] K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.

[11] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *CVPR*, 2015.

[12] Z. Tian, C. Shen, H. Chen, and T. He. Fcos: Fully convolutional one-stage object detection. In *CVPR*, 2019.

[13] Z. Tian, C. Shen, H. Chen, and T. He. Fcos: A simple and strong anchor-free object detector. *TPAMI*, 2020.

[14] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.

[15] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T. Liu. On layer normalization in the transformer architecture. In *ICML*, 2020.